Universidad de Lima

Facultad de Ingeniería

Redes de Computadoras

PROYECTO FINAL

Sistema de Monitoreo y Alerta de Operatividad para Infraestructura de Comunicaciones Estatales con Raspberry Pi y Telegram

Primer avance escrito – Sección 652

Rodriguez Melendez, Samantha Camila
Código 20232548

Ruiz Delgado, Melissa Alexandra
Código 20232637

Tejada Montesinos, Martin Rodrigo
Código 20232907

Valderrama Cruz, Nícolas
Código 20233030

Profesor

George Edwin Romero Velazco

Lima – Perú

Junio del 2025

ÍNDICE

1. RESUMEN.....3
PALABRAS CLAVES..... 4
2. INTRODUCCIÓN.....5
3. ESTADO DEL ARTE.....6
4. MARCO TEÓRICO..... 7
4.1 Raspberry Pi..... 7
4.2 GNS3..... 7
4.3 Protocolos de verificación de estado..... 7
4.4 Telegram Bots.....8
5. IMPLEMENTACIÓN.....10
5.1. SIMULACIÓN DE RED..... 10
5.2. SISTEMA DE ALERTA..... 11
5.2.1 Instalación del entorno virtual y dependencias..... 11
5.2.2 Creación del bot de Telegram.....12
5.2.3 Desarrollo del script alerta_sistema.py..... 13
5.2.4 Ejecución del sistema de monitoreo.....13
5.2.5 Pruebas del sistema..... 13
ANÁLISIS DE RESULTADOS.....18
CONCLUSIONES.....19
REFERENCIAS..... 20
ANEXOS..... 22

ÍNDICE DE FIGURAS

Figura 5.1: Red simulada en GNS3.....10
Figura 5.2: Mapeo de dispositivos y conexiones IP.....10
Figura 5.3: Acceso a RPI vía SSH.....11
Figura 5.4: Ruta estática agregada para alcanzar la red .60 (simulada)..... 11
Figura 5.5: Creación de directorio de trabajo, entorno virtual aislado e instalación dependencias.....11
Figura 5.6: Creación del bot de Telegram..... 12
Figura 5.7: Respuesta de la API de Telegram para obtener el chat_id del usuario..... 12
Figura 5.8: Script de Python para el monitoreo de dispositivos y envío de alertas mediante Telegram.....13
Figura 5.9: Ejecución del sistema..... 13
Figura 5.10: Router desconectado.....14
Figura 5.11: Router conectado nuevamente..... 14
Figura 5.12 Solo PC1 desconectado.....15
Figura 5.13: PC1 conectado nuevamente.....15
Figura 5.14: Solo PC2 desconectado.....16
Figura 5.15: PC2 conectado nuevamente.....16
Figura 5.16: PC1 y PC2 desconectados..... 17
Figura 5.17: PC1 y PC2 conectados nuevamente..... 17
Figura A : Especificaciones de la máquina virtual.....22

1. RESUMEN

Este proyecto propone el diseño e implementación de un sistema de alerta de operatividad para los equipos de comunicaciones de una entidad estatal simulada mediante GNS3. Se utiliza una Raspberry Pi como dispositivo de bajo costo encargado de supervisar el estado de los equipos de red. A través de un script desarrollado en Python, la Raspberry Pi envía comandos de verificación (pings) de forma periódica a los routers y equipos definidos en la red. Si alguno de los dispositivos no responde, el sistema genera una alerta automática que se envía mediante la plataforma de mensajería Telegram a un canal o usuario previamente configurado. Esta funcionalidad permite identificar en tiempo real fallas o desconexiones, lo que facilita una intervención oportuna para restablecer los servicios. La propuesta combina herramientas accesibles y de fácil implementación con plataformas modernas de comunicación, demostrando que es posible fortalecer la infraestructura tecnológica estatal sin recurrir a soluciones costosas. Además, la solución se alinea con el ODS 9, que promueve la innovación y la mejora de infraestructuras, contribuyendo a la digitalización eficiente del sector público.

PALABRAS CLAVES

Raspberry Pi, sistema de alerta, monitoreo de red, Telegram bot, Python, Red LAN simulada, GNS3, equipos de comunicaciones, conectividad, supervisión remota, verificación de operatividad, Automatización, Detección de fallos

2. INTRODUCCIÓN

El monitoreo y la gestión eficiente de redes se han convertido en elementos clave para el funcionamiento confiable de infraestructuras tecnológicas, tanto en entornos educativos como empresariales. Ante este contexto, el presente proyecto desarrolla un sistema automatizado de monitoreo de red y envío de alertas, utilizando una Raspberry Pi, simulaciones en GNS3 y un bot de Telegram como canal de notificación.

Este sistema tiene como propósito supervisar de manera continua la conectividad de dispositivos dentro de una red y alertar al usuario ante posibles caídas o reconexiones. La implementación combina recursos físicos (Raspberry Pi) con entornos virtuales (GNS3), y aprovecha el lenguaje de programación Python junto con la API de Telegram para lograr una solución funcional, de bajo costo y fácilmente escalable.

El trabajo se alinea con el Objetivo de Desarrollo Sostenible (ODS) N.° 9: Industria, innovación e infraestructura, al promover el uso de tecnologías digitales accesibles para mejorar los sistemas de monitoreo, aumentar la eficiencia operativa y fomentar el desarrollo de soluciones tecnológicas sostenibles. Asimismo, busca fortalecer capacidades en automatización, redes y programación, alineadas con las demandas de una infraestructura inteligente e interconectada.

3. ESTADO DEL ARTE

El uso de dispositivos de bajo costo para el monitoreo de redes ha ganado protagonismo en los últimos años, destacando la Raspberry Pi como una herramienta versátil y accesible. Según NetBeez (2023), la Raspberry Pi puede configurarse como un sensor de monitoreo activo en entornos distribuidos, ejecutando tareas como verificación de conectividad (ping), análisis de latencia, y pruebas HTTP. Esta aproximación permite obtener diagnósticos más precisos y en tiempo real sobre el estado de la red desde distintos puntos geográficos, optimizando la detección de fallas.

Por otro lado, la automatización de alertas mediante plataformas de mensajería se ha vuelto una alternativa eficaz para la supervisión remota. OPC Router (2023) describe cómo los bots de Telegram ofrecen una interfaz poderosa y sencilla para integrar sistemas de notificación automatizada con scripts de Python. Esta integración facilita el envío de alertas en tiempo real hacia usuarios o canales específicos, aumentando la capacidad de respuesta ante incidentes de red.

Para simular entornos de red reales, herramientas como GNS3 resultan fundamentales. La documentación oficial de GNS3 (2024) expone que esta plataforma permite emular redes completas con routers, switches y servidores virtuales, siendo una alternativa robusta para pruebas y prototipos. Además, su capacidad de interacción con dispositivos reales, como la Raspberry Pi, habilita escenarios de prueba altamente realistas sin incurrir en altos costos de infraestructura.

La combinación de estas tecnologías —Raspberry Pi, bots de Telegram y GNS3— constituye una base sólida y actual para el desarrollo de soluciones de monitoreo automatizado en redes de comunicación, como la que se plantea en este proyecto.

4. MARCO TEÓRICO

El desarrollo de un sistema de monitoreo y alerta de operatividad requiere conocer a profundidad las herramientas y tecnologías que lo conforman. Elementos como la Raspberry Pi, plataformas de simulación como GNS3 y bots de mensajería brindan una base sólida para la implementación del proyecto. A continuación, se detallan sus funciones.

4.1 Raspberry Pi

La Raspberry Pi se presenta como la base del sistema de monitoreo y alerta debido a su versatilidad y facilidad de integración. Como señalan Soto et al. (2024), sus características de bajo consumo energético y portabilidad la convierten en una opción ideal para soluciones de automatización en entornos donde se requieren dispositivos capaces de operar de forma continua y autónoma. Mediante un script desarrollado en Python, se encarga de supervisar el estado operativo de la red utilizando comandos de verificación, en caso de fallos en la conectividad, enviará alertas automáticas como notificaciones a Telegram con el propósito de centralizar los registros en un chat. En el desarrollo preliminar de la solución, se utilizará Ubuntu para replicar su funcionalidad y verificar su comportamiento antes de implementarlo directamente en la Raspberry Pi.

4.2 GNS3

Para la implementación del sistema, se optó por el entorno de simulación Graphical Network Simulator-3 (GNS3). Esta herramienta permite construir redes virtuales conformadas por routers, switches y otros dispositivos, lo que facilita la recreación de escenarios físicos de manera accesible. A diferencia de otros simuladores, GNS3 está diseñado para simular entornos de red realistas, ya que posibilita la integración con hardware externo como la Raspberry Pi, lo que hace posible realizar pruebas más precisas y representativas del desempeño real del sistema (GNS3 Documentation, 2024). En el presente proyecto, la topología propuesta está compuesta por una nube, un router, un switch y dos PCs.

4.3 Protocolos de verificación de estado

Para garantizar la operatividad continua de la red, es fundamental emplear protocolos que permitan comprobar tanto la conectividad como la disponibilidad de los servicios. Estas

herramientas son esenciales para los sistemas de monitoreo automatizados, ya que juntos recopilan datos de los dispositivos conectados y servicios en ejecución, lo que ayuda a tener una mejor estabilidad en la red (Spasojevic, 2024).

- **ICMP - Internet Control Message Protocol**

  ICMP es utilizado principalmente para identificar y reportar errores en las comunicaciones de red. Mediante el envío de mensajes se puede identificar si un elemento de la red es accesible, detectar caídas de conexión y medir el tiempo de respuesta, todo esto sin necesidad de acceder directamente a los dispositivos. Según Kentik (s.f), entre las herramientas de diagnóstico que lo utilizan se destaca *traceroute* y *ping*, este último envía paquetes de solicitud a la dirección IP determinada; si se recibe una respuesta, significa que el dispositivo está disponible (Kentik, s.f).

- **TCP - Transmission Control Protocol**

  Aunque en la implementación actual no se emplea TCP directamente, su uso es relevante en escenarios donde se requiere verificar la disponibilidad de servicios específicos (como servidores web, bases de datos u otros servicios en red). A diferencia de ICMP, TCP permite no solo comprobar si un dispositivo responde, sino también si un puerto o servicio particular está activo. En futuras mejoras del sistema, podría integrarse TCP para verificar, por ejemplo, si un servidor de aplicaciones está disponible, elevando el nivel de precisión del monitoreo (Metoro, s.f.).

En conjunto, la integración de estos protocolos en sistemas de monitoreo permite automatizar la detección de fallos, generar alertas oportunas a través del bot de Telegram, y asegurar una supervisión constante de la red. En este proyecto, ICMP ha sido implementado mediante scripts en Python que se ejecutan desde la Raspberry Pi conectada a la red simulada en GNS3, logrando una verificación continua del estado operativo de los dispositivos monitoreados.

4.4 Telegram Bots

Los bots de Telegram pueden ser programados para enviar mensajes predeterminados, responder a comandos, conectarse con otros sistemas y procesar datos mediante su API abierta. Según Zomro (2023), estos bots son de gran utilidad para tareas de monitoreo, ya que permiten notificar al usuario cada que suceda un evento específico, sin necesidad de

intervención manual. Esto los convierte en herramientas versátiles para distintos entornos, incluyendo sistemas de supervisión de redes.

Por ello, para el desarrollo del trabajo se utilizará un bot de Telegram como medio de notificación sobre el estado de la red. Cada vez que detecte una conexión establecida o una falla, el bot envía automáticamente un mensaje de alerta al usuario. De esta forma, se garantiza que el control sea continuo y que las actualizaciones del sistema lleguen en tiempo real, sin requerir una vigilancia constante por parte del usuario.

5. IMPLEMENTACIÓN

5.1. SIMULACIÓN DE RED

Se empleó GNS3 para simular la red de una entidad estatal, siguiendo la siguiente topología:

Figura 5.1: Red simulada en GNS3

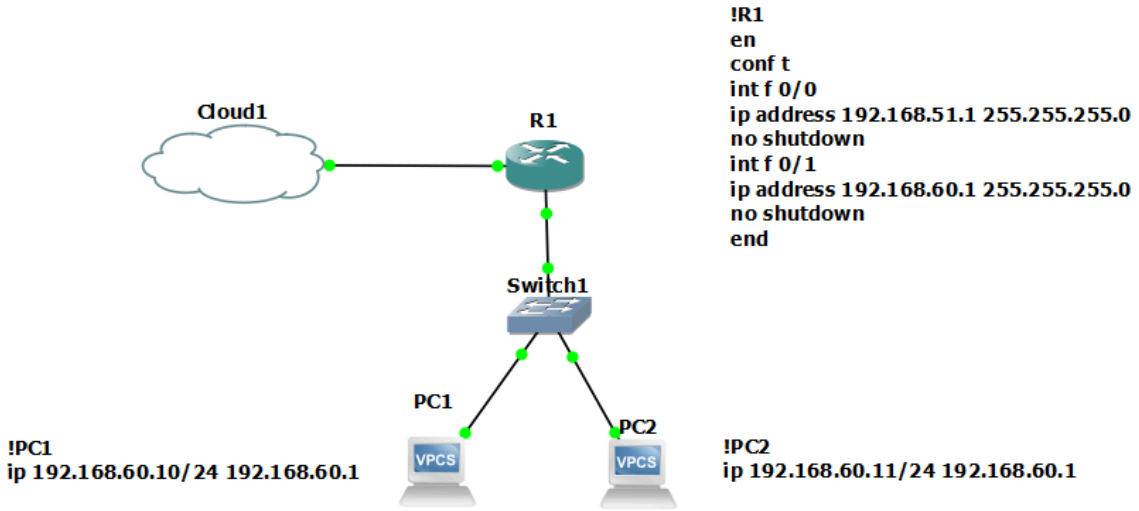


Figura 5.2: Mapeo de dispositivos y conexiones IP

Dispositivo	Interfaz	Dirección
Raspberry Pi	eth0	192.168.51.236
Router GNS3	FastEthernet0/0	192.168.51.1
	FastEthernet0/1	192.168.60.1
PC1	-	192.168.60.10
	Gateway	192.168.60.1
PC2	-	192.168.60.11
	Gateway	192.168.60.1

Como la Raspberry Pi se encuentra únicamente conectada a la red 192.168.51.0/24, fue necesario agregar una ruta estática que le permitiera alcanzar la red 192.168.60.0/24, donde se ubican los dispositivos simulados PC1 y PC2. Esta ruta indica que todo el tráfico destinado a la red 192.168.60.0 debe ser enviado a través del router GNS3, cuya interfaz FastEthernet0/0 tiene la dirección IP 192.168.51.1.

Para realizar esta configuración, primero se accedió remotamente a la Raspberry Pi utilizando el protocolo SSH, desde una laptop conectada a la misma red.

Figura 5.3: Acceso a RPI vía SSH

```
PS C:\Users\aUreLi0> ssh pi@192.168.51.236
pi@192.168.51.236's password:
Linux raspberrypi-16b 6.6.51+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.6.51-1+rpt3 (2024-10-08) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Jul  3 09:04:13 2025 from 192.168.51.151

SSH is enabled and the default password for the 'pi' user has not been changed.
This is a security risk - please login as the 'pi' user and type 'passwd' to set a new password.

pi@raspberrypi-16b:~ $ |
```

Una vez dentro del sistema, se ejecutó el siguiente comando para agregar la ruta:

Figura 5.4: Ruta estática agregada para alcanzar la red .60 (simulada)

```
pi@raspberrypi-16b:~ $ sudo ip route add 192.168.60.0/24 via 192.168.51.1
```

5.2. SISTEMA DE ALERTA

5.2.1 Instalación del entorno virtual y dependencias

Para aislar las dependencias del proyecto, se creó un entorno virtual de Python con ayuda de los siguientes comandos:

Figura 5.5: Creación de directorio de trabajo, entorno virtual aislado e instalación dependencias

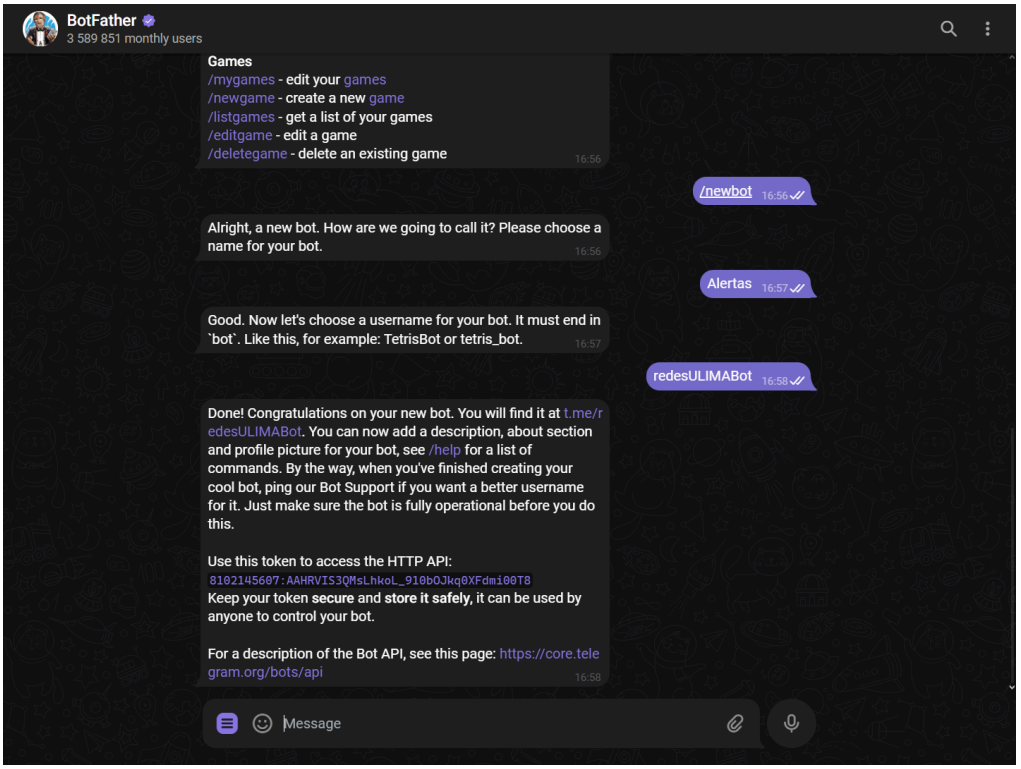
```
pi@raspberrypi-16b:~ $ mkdir ~/proyecto_alertas
pi@raspberrypi-16b:~ $ cd ~/proyecto_alertas
pi@raspberrypi-16b:~/proyecto_alertas $
pi@raspberrypi-16b:~/proyecto_alertas $ sudo apt update
pi@raspberrypi-16b:~/proyecto_alertas $ sudo apt install python3-venv python3-pip -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-venv is already the newest version (3.11.2-1+b1).
python3-pip is already the newest version (23.0.1+dfsg-1+rpt1).
0 upgraded, 0 newly installed, 0 to remove and 324 not upgraded.
pi@raspberrypi-16b:~/proyecto_alertas $ python3 -m venv venv

pi@raspberrypi-16b:~/proyecto_alertas $
pi@raspberrypi-16b:~/proyecto_alertas $ source venv/bin/activate
(venv) pi@raspberrypi-16b:~/proyecto_alertas $ pip install telepot
```

5.2.2 Creación del bot de Telegram

El bot fue creado con ayuda de @BotFather. Una vez creado, se obtuvo el token de autenticación.

Figura 5.6: Creación del bot de Telegram



Para obtener el chat_id, se accedió al siguiente enlace proporcionado por la API de Telegram:https://api.telegram.org/bot8102145607:AAHRVIS3QMsLhkoL_910bOJkq0XFdmi00T8/getUpdates. Aparecerá algo como esto:

Figura 5.7: Respuesta de la API de Telegram para obtener el chat_id del usuario.


```
Impresión con formato estilístico ☑

{
  "ok": true,
  "result": [
    {
      "update_id": 806792504,
      "message": {
        "message_id": 1,
        "from": {
          "id": 6541264899,
          "is_bot": false,
          "first_name": ".",
          "language_code": "es"
        },
        "chat": {
          "id": 6541264899,
          "first_name": ".",
          "type": "private"
        },
```

5.2.3 Desarrollo del script alerta_sistema.py

El script alerta_sistema.py fue diseñado para monitorear continuamente el estado de conexión de los dispositivos mediante ping, enviando alertas por Telegram cuando un equipo cae o se recupera.

Figura 5.8: Script de Python para el monitoreo de dispositivos y envío de alertas mediante Telegram

```
pi@raspberrypi-16b: ~/proye
  GNU nano 7.2                                   alerta_sistema.py
import os
import time
import telepot

TOKEN = '8102145607:AAHRVIS3QMsLhkoL_910bOJkq0XFdmi00T8'
CHAT_ID = '6541264899'
bot = telepot.Bot(TOKEN)

ips = {
    "Router GNS3": {"ip": "192.168.51.1", "estado": True},
    "PC1": {"ip": "192.168.60.10", "estado": True},
    "PC2": {"ip": "192.168.60.11", "estado": True}
}

def esta_vivo(ip):
    return os.system(f"ping -c 1 -W 1 {ip} > /dev/null 2>&1") == 0

def monitorear():
    print("🔍 Sistema de monitoreo activo...")
    while True:
        for nombre, datos in ips.items():
            ip = datos["ip"]
            estado_anterior = datos["estado"]
            en_linea = esta_vivo(ip)

            if en_linea and not estado_anterior:
                bot.sendMessage(CHAT_ID, f"✅ *{nombre}* volvió a estar *en línea*.", parse_mode="Markdown")
            elif not en_linea and estado_anterior:
                bot.sendMessage(CHAT_ID, f"🚨 *{nombre}* está *caído*.", parse_mode="Markdown")
            datos["estado"] = en_linea
        time.sleep(3)

monitorear()

                                                [ Read 33 lines ]
^G Help    ^O Write Out  ^W Where Is   ^K Cut     ^T Execute    ^C Location  M-U Undo    M-A Set Mark  M-] To Bracket
^X Exit    ^R Read File  ^\ Replace    ^U Paste   ^J Justify    ^/ Go To Line M-E Redo    M-6 Copy      ^Q Where Was
```

5.2.4 Ejecución del sistema de monitoreo

Con el entorno virtual activado, se ejecutó el sistema con:

Figura 5.9: Ejecución del sistema

```
pi@raspberrypi-16b:~ $ cd proyecto_alertas/
pi@raspberrypi-16b:~/proyecto_alertas $ source venv/bin/activate
(venv) pi@raspberrypi-16b:~/proyecto_alertas $ python alerta_sistema.py
🔍 Sistema de monitoreo activo...
```

El programa muestra en consola el estado del monitoreo y envía mensajes

5.2.5 Pruebas del sistema

Finalmente, se realizaron pruebas desconectando y reconectando dispositivos en la topología de GNS3. El sistema detectó correctamente las caídas y recuperaciones, enviando notificaciones al bot de Telegram, validando así su funcionamiento.

Pruebas realizadas:

1)

Figura 5.10: Router desconectado

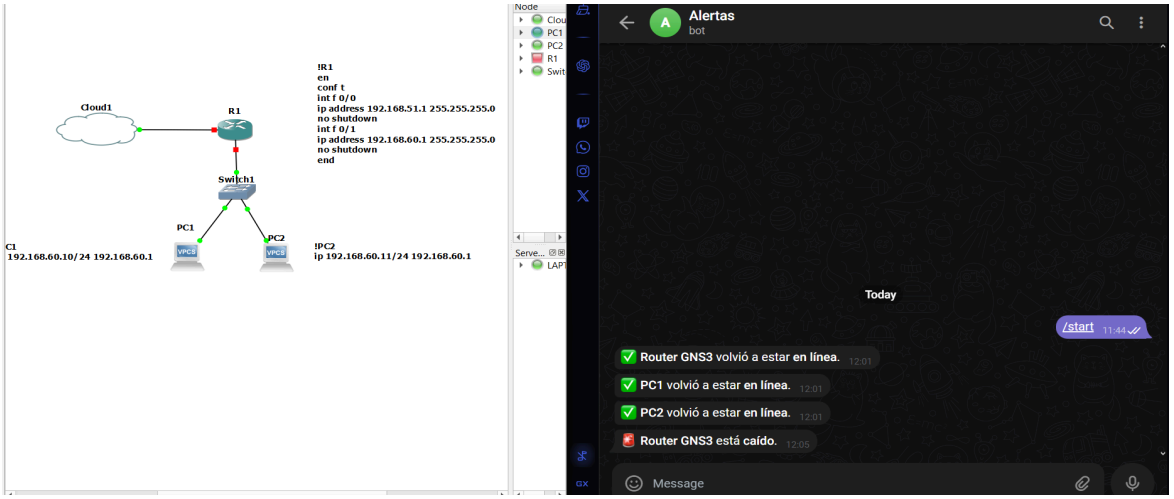
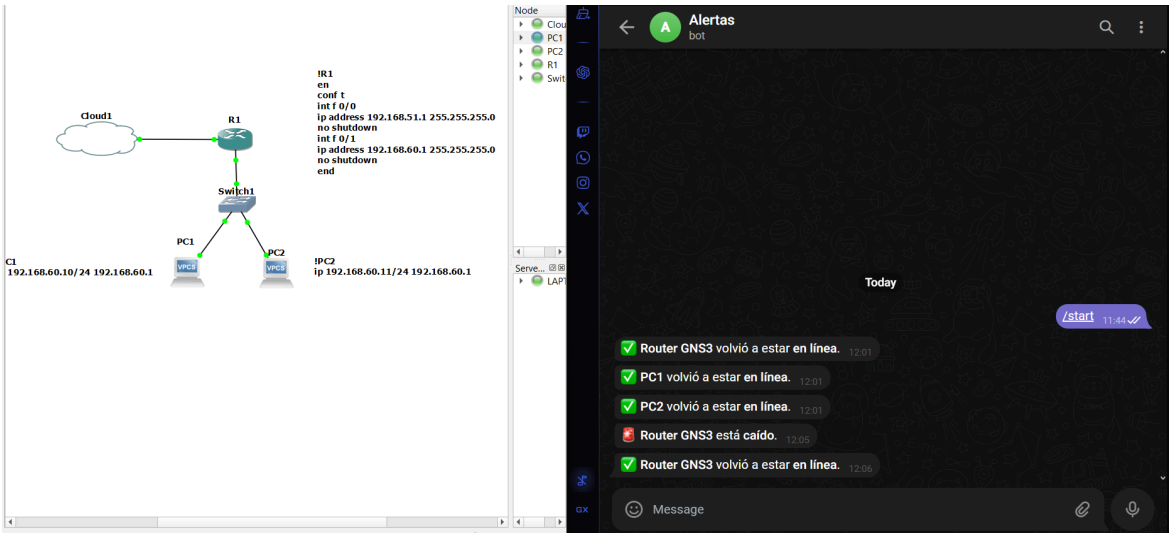


Figura 5.11: Router conectado nuevamente



2)

Figura 5.12 Solo PC1 desconectado

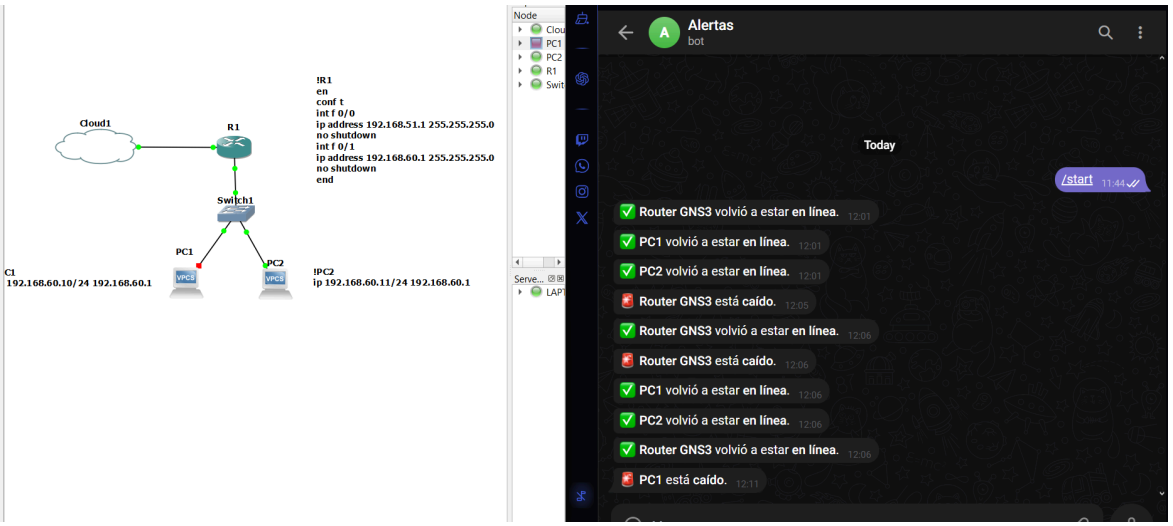
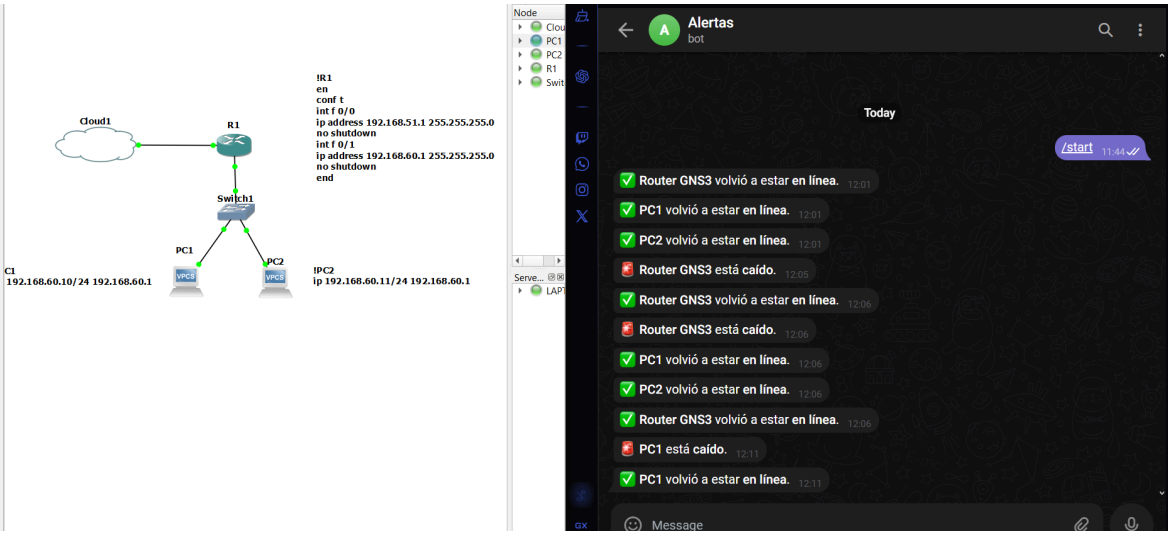


Figura 5.13: PC1 conectado nuevamente



3)

Figura 5.14: Solo PC2 desconectado

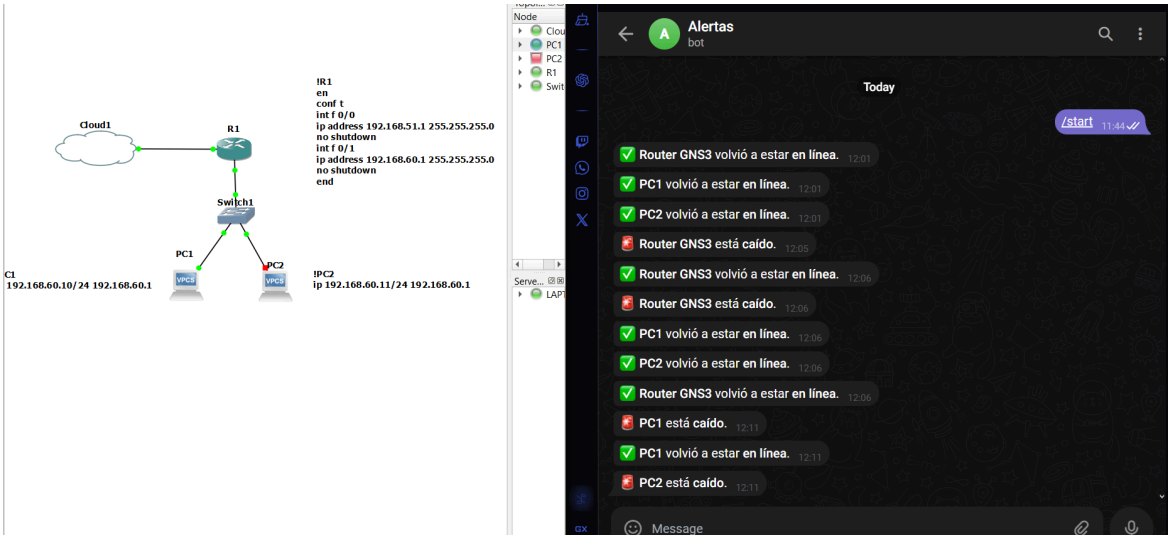
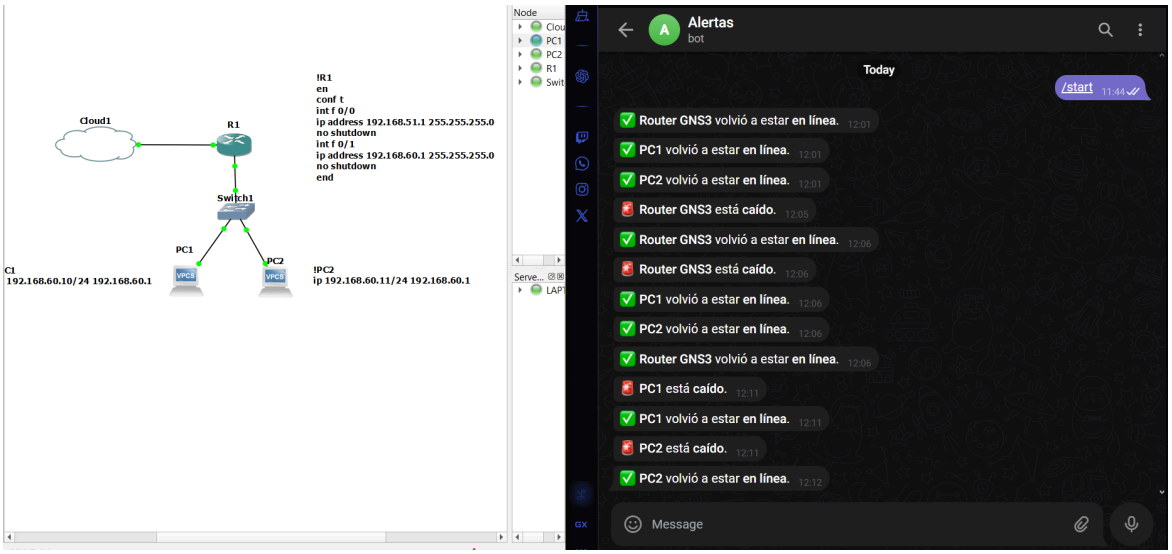


Figura 5.15: PC2 conectado nuevamente



4)

Figura 5.16: PC1 y PC2 desconectados

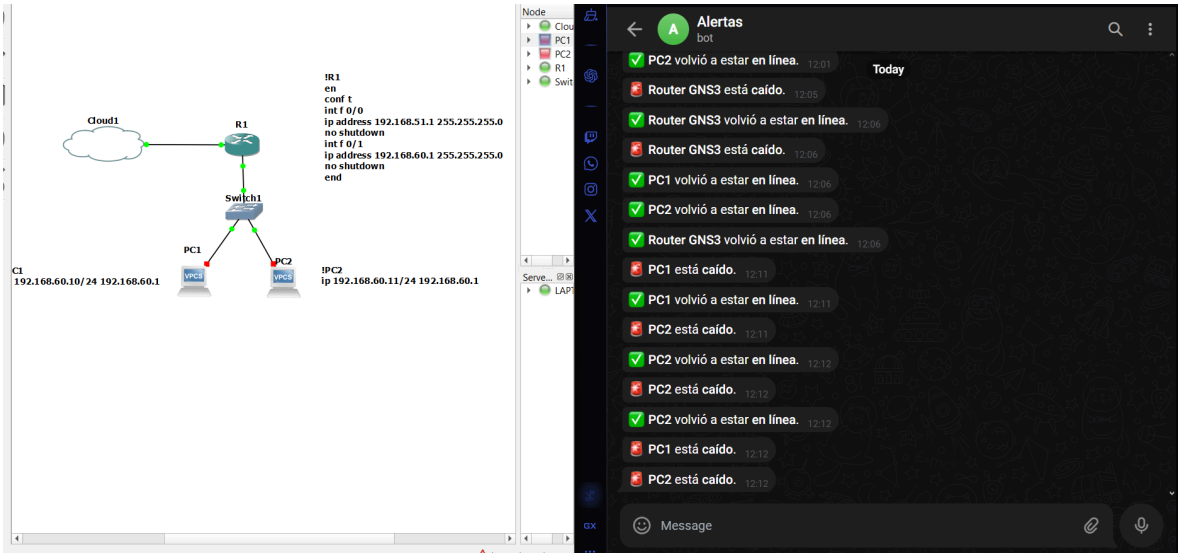
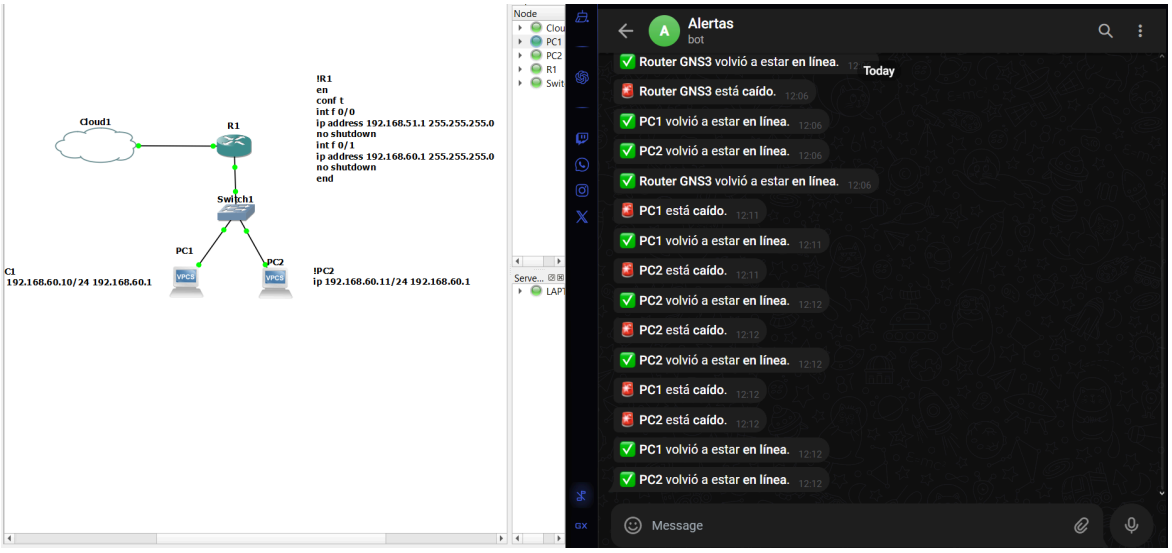


Figura 5.17: PC1 y PC2 conectados nuevamente



ANÁLISIS DE RESULTADOS

Durante las pruebas realizadas, se comprobó que la Raspberry Pi pudo comunicarse correctamente con los dispositivos simulados en GNS3 a través del router, gracias a la configuración adecuada de direcciones IP, rutas estáticas y la habilitación del reenvío de paquetes. Esta configuración fue fundamental para que el sistema pudiera operar en un entorno de red híbrido, conectando una máquina física con una red simulada.

El script en Python, ejecutado desde la Raspberry Pi, realizó pings periódicos a las direcciones IP de PC1 y PC2, a su vez que al Router. Ante cualquier interrupción en la conectividad —ya sea por apagado de los equipos, errores de configuración o desconexiones momentáneas— el sistema respondió de forma inmediata, enviando alertas al usuario a través de un bot de Telegram previamente configurado. Este comportamiento validó la funcionalidad del sistema de monitoreo en tiempo real.

No obstante, se identificaron ciertos casos de falsos positivos, generados por pérdidas momentáneas de conectividad o retardos en la respuesta de los dispositivos simulados. Estos eventos hicieron evidente la necesidad de una lógica de verificación más robusta, que no dependa de un único intento fallido para emitir una alerta. Una mejora concreta sería implementar un sistema que requiera múltiples fallos consecutivos antes de considerar una caída real, lo cual reduciría notificaciones innecesarias y aumentaría la confiabilidad del sistema.

La integración entre una red simulada (GNS3) y un entorno físico (Raspberry Pi) también permitió evaluar la solidez del sistema en situaciones reales. Se observaron los efectos prácticos de una mala configuración de IPs, gateways o rutas, y cómo estos errores impedían la conectividad. Resolver estas situaciones requirió aplicar conocimientos técnicos sobre direccionamiento IP, enrutamiento estático, entre otros, consolidando habilidades fundamentales en la administración de redes.

En conjunto, los resultados obtenidos mostraron que el sistema no solo funcionó como se esperaba, sino que también sirvió como una plataforma de aprendizaje práctico, revelando tanto sus fortalezas como las oportunidades de mejora para futuras versiones.

CONCLUSIONES

El proyecto logró cumplir satisfactoriamente su objetivo principal: desarrollar e implementar un sistema de monitoreo de red que, mediante una Raspberry Pi, permita detectar desconexiones de dispositivos y enviar alertas automáticas a través de Telegram. La integración de herramientas como Python, GNS3 y el uso de un bot en Telegram permitió construir una solución funcional, simple y efectiva, capaz de operar de forma continua sin intervención manual.

La elección de un entorno virtual (venv) en Python resultó adecuada, ya que permitió instalar las dependencias necesarias sin afectar el entorno global del sistema operativo. Además, el uso de la librería telepot facilitó la comunicación directa con la API de Telegram, permitiendo un flujo inmediato de notificaciones al usuario ante cualquier evento relevante en la red.

Desde el punto de vista de redes, el proyecto reforzó conocimientos clave como la asignación de IPs estáticas, el uso de gateways, la configuración de rutas estáticas y el reenvío de paquetes. La necesidad de conectar una red simulada (GNS3) con un entorno físico (Raspberry Pi) representó un desafío interesante, que exigió comprender y aplicar conceptos de conectividad real, identificación de subredes, y resolución de problemas.

Si bien el sistema demostró ser funcional, se identificaron aspectos que pueden optimizarse. Uno de los más relevantes es la sensibilidad del monitoreo ante fallos breves o pérdidas de paquetes, lo que puede generar falsos positivos. Para mejorar la confiabilidad del sistema, se sugiere implementar un esquema de verificación por múltiples intentos fallidos antes de confirmar una caída.

Finalmente, este proyecto no solo cumplió con los requerimientos técnicos planteados, sino que también aportó una valiosa experiencia práctica en la integración de software, redes y automatización. Su carácter escalable permite pensar en futuras extensiones, como la incorporación de una interfaz web, monitoreo de más dispositivos o incluso la implementación de un sistema de priorización de alertas. En conjunto, representa una base sólida para continuar explorando soluciones inteligentes de administración de red.

REFERENCIAS

Fortinet. (s.f) *¿Qué es TCP/IP en redes?*. Recuperado de https://www.fortinet.com/lat/resources/cyberglossary/tcp-ip

Giddi. (2023). *Monitoring servers using Telegram*. Recuperado de https://giddi.net/posts/monitoring-servers-using-telegram/

GNS3 Docs. (2024). *GNS3 Documentation*. Recuperado de https://docs.gns3.com/docs/

How to SSH into Raspberry Pi. (n.d.). https://tailscale.com/learn/how-to-ssh-into-a-raspberry-pi

Kentik. (s.f). *Network monitoring protocols: 6 essential network technologies.* Recuperado de https://www.kentik.com/kentipedia/network-monitoring-protocols/

Metoro Documentation. (s.f). *TCP port monitoring*. https://metoro.io/docs/uptime-monitoring/tcp-port-monitoring

NetBeez. (2023). *Raspberry Pi Network Monitoring*. Recuperado de https://netbeez-net.translate.goog/blog/raspberry-pi-network-monitoring

OPC Router. (2023). *¿Qué es un bot de Telegram?*. https://www-opc--router-com.translate.goog/what-is-a-telegram-bot

Python Software Foundation. (n.d.) *os — Miscellaneous operating system interfaces*. Python documentation. https://docs.python.org/3/library/os.html

Python Software Foundation. (n.d.) *time — Time access and conversions*. Python documentation. https://docs.python.org/3/library/time.html

Raspberry Pi Foundation. (n.d) *Remote access*. Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/computers/remote-access.html

Soto, J., Peña, J., Ariza, J., & Gil, S. (2024). Low-Cost Raspberry Pi-Based Remote Laboratory for IoT, LoRa and Python Programming Education. *2024 XVI Congreso de Tecnología, Aprendizaje y Enseñanza de la Electrónica (TAEE)*, Málaga, España. https://doi.org/10.1109/TAEE59541.2024.10604947

Spasojevic, A. (2024). ¿Qué es el monitoreo de red? phoenixNAP IT Glossary. https://phoenixnap.mx/glosario/%C2%BFQu%C3%A9-es-el-monitoreo-de-red%3F

telepot 12.7 documentation. (n.d.) *Read the Docs*. https://telepot.readthedocs.io/en/latest/

Zomro. (2023). *How to create a Telegram bot for monitoring remote hosts*. https://zomro.com/blog/faq/316-kak-sozdat-telegram-bota-dlja-monitoringa-udalennyh-hostov

ANEXOS

Anexo A: Implementación preliminar del Sistema de Monitoreo y Alerta de Operatividad con Raspberry Pi y Telegram

Antes de realizar la implementación final sobre la Raspberry Pi física, se llevó a cabo una etapa preliminar del proyecto en un entorno virtualizado. Para ello, se utilizó una máquina virtual con Ubuntu, configurada para simular el comportamiento y las condiciones operativas del sistema que posteriormente se desplegará en la Raspberry Pi.

El propósito principal de esta implementación preliminar fue validar la lógica del script en Python, el funcionamiento del bot de Telegram y la capacidad del sistema para detectar eventos de red de forma efectiva.

El comportamiento del sistema fue prácticamente idéntico al entorno final. La única variación significativa fue la forma de conectar ambos entornos (VM y GNS3). En esta etapa, la máquina virtual se integró a la red de GNS3 utilizando adaptadores de red personalizados.

Figura A : Especificaciones de la máquina virtual

Nota: Se empleó VMnet1 para la comunicación en modo host-only entre la máquina virtual y el host, mientras que VMnet8 se utilizó para proporcionar acceso a Internet mediante NAT.

Gracias a esta implementación preliminar, fue posible reducir significativamente los errores en la fase final sobre la Raspberry Pi real y asegurar que el sistema respondiera de forma estable en el entorno híbrido (físico y simulado).

Anexo B: Script de Python

```
import os
import time
import telepot

TOKEN = '8102145607:AAHRVIS3QMsLhkoL_910bOJkq0XFdmi00T8'
CHAT_ID = '6541264899'
bot = telepot.Bot(TOKEN)

ips = {
    "Router GNS3": {"ip": "192.168.51.1", "estado": True},
    "PC1": {"ip": "192.168.60.10", "estado": True},
    "PC2": {"ip": "192.168.60.11", "estado": True}
}

def esta_vivo(ip):
    return os.system(f"ping -c 1 -W 1 {ip} > /dev/null 2>&1") == 0

def monitorear():
    print("🔍 Sistema de monitoreo activo...")
    while True:
        for nombre, datos in ips.items():
            ip = datos["ip"]
            estado_anterior = datos["estado"]
            en_linea = esta_vivo(ip)

            if en_linea and not estado_anterior:
                bot.sendMessage(CHAT_ID, f"✅ *{nombre}* volvió a estar *en línea*.",
parse_mode="Markdown")
            elif not en_linea and estado_anterior:
                bot.sendMessage(CHAT_ID, f"🚨 *{nombre}* está *caído*.",
parse_mode="Markdown")
            datos["estado"] = en_linea
        time.sleep(3)
monitorear()
```